



**Diseño e implementación de la extensión 'C'
(instrucciones comprimidas) de
RISC-V en un procesador pipelined**

Integrantes:

APELLIDOS, NOMBRES	Participación
Thiago Gabriel Rivera Matta	100%
Jesús Valentín Niño Castañeda	100%
Reyes Medina, Jordinn Martin	100%

ASIGNATURA:

Arquitectura de Computadoras

DOCENTE:

Carlos Williams

ÍNDICE

1. Implementación de las Instrucciones.....	3
1.1 Instrucciones Implementadas.....	3
1.2 Cambios del Código.....	4
1.2.1 El Módulo Descompresor (decompressor.v).....	4
1.2.2 Adaptación del Datapath y Control del PC (datapath.v).....	5
1.2.3 Fetch Desalineado en la Memoria de Instrucciones (imem.v).....	6
1.2.4 Limpieza y Bloques de Ejecución (aludec.v y pipereg.v).....	6
1.2.5 Monitor Universal de Pruebas (testbench.v).....	7
1.3 Explicación de las Instrucciones.....	7
c.addi.....	7
c.add.....	8
c.sub, c.xor, c.or, c.and.....	9
c.slli.....	10
c.srli, c.srai.....	11
c.lui.....	11
2. Resultados.....	12
2.1 Programa ISA.....	12
2.2 Validación de cada una de las instrucciones.....	14
Validacion de Prueba Aritmética (c.addi, c.add, c.sub):.....	14
Validacion de Prueba Logica (c.and, c.or):.....	15
Validación Prueba de Desplazamientos y XOR:.....	16
Validación Prueba LUI y General:.....	16
3. Bibliografía.....	17

1. Implementación de las Instrucciones

1.1 Instrucciones Implementadas

La implementación de la extensión C incluye las instrucciones comprimidas requeridas para ejecutar correctamente los programas de prueba. En el cuadro siguiente se resumen las instrucciones soportadas y su función principal.

Instrucción	Formato	Operación principal	Uso en el proyecto
<code>c.addi</code>	I-type	Suma inmediata	Inicialización de registros y pruebas de datos
<code>c.add</code>	R-type	Suma de registros	Validación de forwarding y ejecución aritmética
<code>c.sub</code>	R-type	Resta de registros	Verificación de operaciones aritméticas y comparaciones
<code>c.and</code>	R-type	And entre registros	Validación de operaciones lógicas entre registros
<code>c.or</code>	R-type	Or entre registros	Validación de operaciones lógicas entre registros
<code>c.xor</code>	R-type	Xor entre registros	Validación de operaciones lógicas entre registros
<code>c.slli</code>	I-type	Shift izquierdo lógico inmediato	Verificación de operaciones de

Instrucción	Formato	Operación principal	Uso en el proyecto
			desplazamiento con inmediatos
<code>c.srli</code>	I-type	Shift derecho lógico inmediato	Verificación de desplazamiento lógico hacia la derecha
<code>c.srai</code>	I-type	Shift derecho aritmético inmediato	Verificación del manejo correcto del bit de signo
<code>c.lui</code>	U-type	Cargar en el intermedio superior	Inicialización eficiente de constantes de 32 bits

1.2 Cambios del Código

1.2.1 El Módulo Descompresor (`decompressor.v`)

El corazón de la extensión 'C' es un traductor combinacional que toma instrucciones de 16 bits y las expande a sus equivalentes de 32 bits del ISA base de RISC-V en tiempo real. Se creó un módulo completamente nuevo (`decompressor.v`) que lee los bits menos significativos (`op`) y los bits de función (`funct3`) para determinar a qué cuadrante pertenece la instrucción comprimida y mapear los campos (como los registros restringidos `x8-x15`) a la codificación estándar de 32 bits.

```
None
module decompressor(
    input wire [15:0] instr_c,
    output reg [31:0] instr_32
);
    wire [1:0] op = instr_c[1:0];
```

```

wire [2:0] funct3 = instr_c[15:13];

// Expansión de registros comprimidos x8-x15
wire [4:0] rs1_c = {2'b01, instr_c[9:7]};
wire [4:0] rs2_c = {2'b01, instr_c[4:2]};

always @* begin
    case (op)
        2'b00: begin /* Cuadrante 0: c.lw, c.sw */ end
        2'b01: begin /* Cuadrante 1: c.addi, c.jal, c.beqz, etc. */ end
        2'b10: begin /* Cuadrante 2: c.slli, c.lwsp, c.add, etc. */ end
        default: instr_32 = 32'h00000013; // NOP de seguridad
    endcase
end
endmodule

```

1.2.2 Adaptación del Datapath y Control del PC (datapath.v)

En el datapath original, el Program Counter (PC) siempre avanzaba sumando 4 bytes (PC + 4). Con la extensión 'C', el salto del PC ahora es dinámico: debe sumar 2 si la instrucción actual es de 16 bits, o 4 si es de 32 bits. Además, se insertó el decompresor en la etapa de Fetch para que evalúe la instrucción entrante. Si la instrucción detectada es comprimida (evaluando si los últimos dos bits son distintos de 11), se pasa al registro de segmentación (IF/ID) la versión descomprimida; de lo contrario, pasa la instrucción original.

```

None
// Detección de instrucción comprimida
wire is_compressed = (InstrF[1:0] != 2'b11);

// Instanciación del decompresor
wire [31:0] InstrF_decomp;
decompressor decomp (.instr_c(InstrF[15:0]), .instr_32(InstrF_decomp));

// Selección de la instrucción final que pasará al pipeline
wire [31:0] InstrF_final = is_compressed ? InstrF_decomp : InstrF;

// Cálculo dinámico del salto del Program Counter
wire [31:0] PCStep = is_compressed ? 32'd2 : 32'd4;
adder pcaddstep(.a(PCF), .b(PCStep), .y(PCPlusStepF));

// Actualización del registro IF/ID con la instrucción final y el PCStep
pipereg r_if_id_instr (.clk(clk), .reset(reset), .en(~StallD),
    .clr(PCSrcE), .d(InstrF_final), .q(InstrD));

```

1.2.3 Fetch Desalineado en la Memoria de Instrucciones (imem.v)

Dado que las instrucciones comprimidas ocupan 2 bytes (16 bits), el PC puede terminar apuntando a direcciones que terminan en 2 (por ejemplo, 0x...002, 0x...006), lo cual rompe el alineamiento estándar de palabras de 32 bits. Si una instrucción de 32 bits empieza en un límite de "half-word" desalineado, cruzará la frontera de la palabra de memoria actual. Para solucionarlo, se modificó la memoria de instrucciones para que concatene los 16 bits superiores de la palabra actual con los 16 bits inferiores de la siguiente palabra cuando detecte que la dirección está desalineada ($a[1] == 1$).

```
None
wire [29:0] word_addr = a[31:2];
      wire      half_aligned = a[1]; // Detecta si termina en 2 (ej. 0x02)

      // Si está desalineado, une la mitad superior de esta palabra con la
      // mitad inferior de la siguiente
      assign rd = half_aligned ? {RAM[word_addr + 1][15:0],
RAM[word_addr][31:16]} : RAM[word_addr];
```

1.2.4 Limpieza y Bloques de Ejecución (aludec.v y pipereg.v)

Para evitar condiciones de carrera ("latches" inferidos) y asegurar un diseño combinacional y secuencial limpio, se refactorizó la lógica en varios archivos envolviendo sentencias en bloques begin ... end. Aunque esto no cambia la arquitectura a nivel macro, garantiza que la síntesis en hardware asigne las señales correctamente. Asimismo, los registros de pipeline (pipereg.v) se limpiaron para manejar el en (Stall) y clr (Flush) de una forma estructurada con prioridad, asegurando que si ocurre un Stall ($en == 0$), el registro simplemente mantenga su valor.

```
None
always @* begin
    case(ALUOp)
        // ...
        2'b10: begin // R-type e I-type ALU
            case(func3)
                3'b000: begin
                    if (func7b5 & opb5) ALUControl = 4'b0001;
                    else                 ALUControl = 4'b0000;
                end
            // ...
        endcase
    end
```

```

        end
        // ...
    endcase
end

```

1.2.5 Monitor Universal de Pruebas (testbench.v)

Finalmente, para validar programas dinámicos que combinan instrucciones ISA y RVC, el testbench se ajustó. Ahora no busca un resultado en una dirección arbitraria sujeta a cambios por la compresión, sino que establece un "STOP UNIVERSAL": se asume que todos los algoritmos de prueba terminarán escribiendo su resultado final en la dirección de memoria 200. Esto permite testear quicksort_rv32c, matrix_rv32c y tree_rv32c sin modificar el testbench cada vez.

```

None
// Monitor Universal de Resultados
always @(negedge clk) begin
    if(MemWrite) begin
        $display("RAM Write: Escribio %d en la direccion %d", WriteData,
DataAdr);

        // CONDICIÓN UNIVERSAL: Termina al escribir en la dir 200
        if(DataAdr === 200) begin
            $display("¡EXITO TOTAL! El programa ha finalizado.");
            $finish;
        end
    end
end
end

```

1.3 Explicación de las Instrucciones

A continuación se detalla la lógica de decodificación de cada instrucción implementada. Los comentarios dentro del código explican de forma explícita cómo se expanden los campos comprimidos para reconstruir la estructura estándar de 32 bits del ISA RISC-V.

c.addi

La instrucción c.addi pertenece al cuadrante 2'b01 con un código de función funct3 igual a 3'b000. Su propósito es sumar un inmediato con signo al registro destino rd, el cual también actúa como el primer registro fuente (rs1).

El proceso de descompresión realiza las siguientes asignaciones para construir la palabra de 32 bits (addi del formato I):

- Inmediato (instr_32[31:20]): Se compone de 12 bits. Se extrae el bit del signo de instr_c[12] y se replica 6 veces para la extensión de signo en los bits superiores, concatenándose con el propio instr_c[12] y el segmento instr_c[6:2].
- Registro Fuente 1 (instr_32[19:15]): Corresponde al campo instr_c[11:7], que identifica al registro operativo.
- Código de función (instr_32[14:12]): Se establece en 3'b000, correspondiente a la operación addi.
- Registro Destino (instr_32[11:7]): Al igual que rs1, se mapea directamente desde instr_c[11:7].
- Opcode (instr_32[6:0]): Se asigna el valor constante 7'b0010011, que define a las operaciones aritméticas de tipo I con inmediato en RISC-V.

None

```
module decompressor(...
);
always @* begin
    case(op)
        ...
        2'b01: begin
            case (funct3)
                3'b000: instr_32 = {{6{instr_c[12]}}, instr_c[12], instr_c[6:2]},
instr_c[11:7], 3'b000, instr_c[11:7], 7'b0010011}; // c.addi
            ...
        end
    endcase
end
```

c.add

Esta instrucción se ubica en el cuadrante 2'b10 con un funct3 de 3'b100. Realiza la suma de dos registros estándar y almacena el resultado en el registro destino, compartiendo la misma restricción donde el registro destino rd actúa también como el primer operando rs1.

Debido a que este subespacio de codificación es compartido con las instrucciones de salto por registro (c.jr y c.jalr), el descompresor aplica filtros condicionales evaluando los bits instr_c[12] e instr_c[6:2] antes de asegurar que se trata de un c.add. La traducción al formato R de 32 bits (add) se estructura de la siguiente manera:

- Funct7 (instr_32[31:25]): Se fija en 7'b0000000 para indicar una suma estándar (sin modificaciones de resta).
- Registro Fuente 2 (instr_32[24:20]): Se extrae directamente del campo de 5 bits en instr_c[6:2].
- Registro Fuente 1 (instr_32[19:15]) y Registro Destino (instr_32[11:7]): Ambos campos se enlazan al registro especificado en instr_c[11:7].
- Funct3 (instr_32[14:12]): Se le asigna 3'b000.
- Opcode (instr_32[6:0]): Se establece en 7'b0110011, correspondiente al opcode de tipo R para operaciones de la ALU entre registros.


```

None
module decompressor(...
);
always @* begin
    case(op)
        ...
        2'b10: begin
            case(funcnt3)
                3'b100: begin
                    if (instr_c[12] == 0) ...
                    else if (instr_c[6:2] == 0) ...
                    else // c.add
                        instr_32 = {7'b0000000, instr_c[6:2], instr_c[11:7], 3'b000,
instr_c[11:7], 7'b0110011};
                ...
            endcase
        end
    endcase
end

```

c.sub, c.xor, c.or, c.and

Este conjunto de instrucciones lógicas y aritméticas se encuentra agrupado bajo el cuadrante 2'b01 y comparte el código `funcnt3 = 3'b100`. A diferencia de las anteriores, estas operaciones utilizan una codificación compacta de registros de 3 bits (`rs1_c` y `rs2_c`), lo que restringe el uso exclusivo a los registros x8 hasta x15 (el bloque de registros más activos de acuerdo a la ABI de RISC-V). El descompresor antepone los bits 2'b01 a estos campos de 3 bits para reconstruir la dirección completa de 5 bits en el banco de registros (wire `rs1_c` y `rs2_c`).

La decodificación interna utiliza los bits `instr_c[11:10] == 2'b11` como selector de operaciones sobre registros, y posteriormente emplea los bits `instr_c[6:5]` como un sub-opcode para discriminar la instrucción exacta:

- 2'b00 (c.sub): Mapea a la instrucción sub de formato R. Configura `funcnt7` en 7'b01000000 (activando el bit de resta) y `funcnt3` en 3'b000.
- 2'b01 (c.xor): Mapea a xor. Configura `funcnt7` en 7'b00000000 y `funcnt3` en 3'b100.
- 2'b10 (c.or): Mapea a or. Configura `funcnt7` en 7'b00000000 y `funcnt3` en 3'b110.
- 2'b11 (c.and): Mapea a and. Configura `funcnt7` en 7'b00000000 y `funcnt3` en 3'b111.

En todos los casos, el opcode destino de 32 bits es el formato R estándar (7'b0110011), el registro fuente 2 es `rs2_c`, y tanto el registro fuente 1 como el registro destino se mapean a `rs1_c`.

```

None
module decompressor(...
);
always @* begin
    case(op)

```

```

...
2'b01: begin
  case(func3)
    3'b100: begin
      case (instr_c[11:10])
        ...
        2'b11: begin
          case (instr_c[6:5])
            2'b00: instr_32 = {7'b0100000, rs2_c, rs1_c, 3'b000, rs1_c,
7'b0110011}; // c.sub
            2'b01: instr_32 = {7'b0000000, rs2_c, rs1_c, 3'b100, rs1_c,
7'b0110011}; // c.xor
            2'b10: instr_32 = {7'b0000000, rs2_c, rs1_c, 3'b110, rs1_c,
7'b0110011}; // c.or
            2'b11: instr_32 = {7'b0000000, rs2_c, rs1_c, 3'b111, rs1_c,
7'b0110011}; // c.and
          endcase
        end
      endcase
    end
  end
...

```

c.slli

Ubicada en el cuadrante 2'b10 con funct3 = 3'b000, c.slli realiza un desplazamiento lógico a la izquierda del registro rd por una cantidad determinada por un inmediato (shamt).

La reconstrucción a una instrucción slli de 32 bits (formato I) opera de la siguiente forma:

- Cantidad de desplazamiento / Shamt (instr_32[24:20]): Se extrae directamente de los bits instr_c[6:2], permitiendo un rango de desplazamiento de 0 a 31 posiciones para la arquitectura RV32.
- Funct7 (instr_32[31:25]): Se rellena con ceros (7'b0000000) especificando que se trata de un corrimiento lógico.
- Registros (rs1 y rd): Se asignan utilizando la dirección de 5 bits presente en instr_c[11:7].
- Funct3 y Opcode: Se configuran en 3'b001 (SLL) y 7'b0010011 (Tipo I ALU) respectivamente.

```

None
module decompressor(...
);
always @* begin
  case(op)
    ...

```

```

2'b10: begin
  case(func3)
    3'b000: instr_32 = {7'b0000000, instr_c[6:2], instr_c[11:7], 3'b001,
instr_c[11:7], 7'b0010011}; // c.slli
    ...

```

c.srli, c.srai

Ambas instrucciones de desplazamiento hacia la derecha se ejecutan en el cuadrante 2'b01 compartiendo el mismo func3 = 3'b100, operando también sobre el conjunto de registros restringidos x8-x15 (rs1_c). La cantidad de posiciones a desplazar (shamt) se lee de instr_c[6:2].

La diferenciación fundamental entre un desplazamiento lógico y uno aritmético se determina mediante los bits de control internos instr_c[11:10]:

- 2'b00 (c.srli): Desplazamiento lógico a la derecha. Traduce a 32 bits asignando 7'b0000000 en el campo func7 para rellenar los espacios vacíos con ceros.
- 2'b01 (c.srai): Desplazamiento aritmético a la derecha. Traduce a 32 bits asignando 7'b0100000 en el campo func7, lo que instruye a la ALU a replicar el bit de signo del operando original en las posiciones vacías.

Ambas instrucciones resultantes adoptan el func3 = 3'b101 (SRL/SRA) y el opcode de tipo I 7'b0010011.

```

None
module decompressor(...
);
always @* begin
  case(op)
    ...
    2'b01: begin
      case(func3)
        3'b100: begin
          case (instr_c[11:10])
            2'b00: instr_32 = {7'b0000000, instr_c[6:2], rs1_c, 3'b101, rs1_c,
7'b0010011}; // c.srli
            2'b01: instr_32 = {7'b0100000, instr_c[6:2], rs1_c, 3'b101, rs1_c,
7'b0010011}; // c.srai
          ...

```

c.lui

Ubicada en el cuadrante 2'b01 con un código de función func3 = 3'b011, la instrucción c.lui carga un inmediato de gran tamaño en la parte superior del registro de destino rd.

El hardware del descompresor implementa una condición de seguridad crítica: si el campo del registro destino `instr_c[11:7]` es igual a `5'b00010`, la instrucción cambia su comportamiento para corresponder a `c.addi16sp` (manipulación del puntero de pila). En caso contrario, se procesa de forma estándar como un `c.lui` mapeando la instrucción al formato U de 32 bits (`lui`):

- Inmediato de 20 bits (`instr_32[31:12]`): Se genera concatenando una extensión del bit de signo `instr_c[12]` replicado 15 veces junto con el fragmento de datos localizados en `instr_c[6:2]`. Esto inicializa eficientemente constantes desplazadas hacia los bits superiores de la palabra.
- Registro Destino (`instr_32[11:7]`): Toma de manera directa los 5 bits del campo de origen en `instr_c[11:7]`.
- Opcode (`instr_32[6:0]`): Adopta el identificador fijo `7'b0110111`, propio de la operación `lui` en RISC-V.

```
None
module decompressor(...
);
always @* begin
    case(op)
        ...
        2'b01: begin
            case(func3)
                3'b011: begin
                    if (instr_c[11:7] == 5'b00010) ...
                    else // c.lui
                        instr_32 = {{15{instr_c[12]}}, instr_c[6:2], instr_c[11:7],
7'b0110111};
                ...
            endcase
        end
    endcase
end
```

2. Resultados

2.1 Programa ISA

Para evaluar la correcta implementación de todas las instrucciones comprendidas en esta entrega e igualmente el ISA hemos diseñado 4 programas de prueba para su validación. La característica principal de estos códigos es la coexistencia simultánea de instrucciones base de 32 bits y comprimidas de 16 bits. Esto obliga a la Unidad de Control y al Program Counter (PC) a adaptarse dinámicamente al tamaño de la palabra extraída de la memoria. Los códigos están distribuidos de la siguiente manera:

1. **Prueba Aritmética:** Intercala sumas y restas clásicas (`addi`, `add`, `sub`) con sus contrapartes comprimidas (`c.addi`, `c.add`, `c.sub`). Demuestra que la ALU procesa correctamente los datos sin importar el formato de compresión de origen.

None

```
00500513 // addi a0, x0, 5 - a0 = 5
00200593 // addi a1, x0, 2 - a1 = 2
952e0505 // [c.add a0, a1] | [c.addi a0, 1] - c.addi: a0 = 5 + 1 = 6 //
c.add: a0 = 6 + 2 = 8
00b50533 // add a0, a0, a1 - a0 = 8 + 2 = 10
8d0d0585 // [c.sub a0, a1] | [c.addi a1, 1] - c.addi: a1 = 2 + 1 = 3 // a0 =
10 - 3 = 7
40b50533 // sub a0, a0, a1 - a0 = 7 - 3 = 4
0ca02423 // sw a0, 200(x0) - guarda el valor 4
0000006f // jal x0, 0
```

2. **Prueba Lógica:** Evalúa el funcionamiento de las compuertas lógicas bit a bit, combinando `and`, `or` de 32 bits con `c.and`, `c.or` de 16 bits.

None

```
00f00513 // addi a0, x0, 15 -> [32 bits] a0 = 15 (Binario: 01111)
00600593 // addi a1, x0, 6 -> [32 bits] a1 = 6 (Binario: 00110)
8d6d0505 // [c.and a0, a1] | [c.addi a0, 1] - c.addi: a0 = 15 + 1 = 16 //
c.and: a0 = 16 & 6 = 0
00f00513 // addi a0, x0, 15 - a0 = 15
8d4d0585 // [c.or a0, a1] | [c.addi a1, 1] - c.addi: a1 = 6 + 1 = 7 //
c.or: a0 = 15 | 7 = 15
00b57533 // and a0, a0, a1 - a0 = 15 & 7 = 7
0ca02423 // sw a0, 200(x0) - guardar el 7
0000006f // jal x0, 0
```

3. **Prueba de Desplazamientos y XOR:** Valida el funcionamiento de los desplazadores lógicos y aritméticos comprimidos (`c.sll`, `c.srl`, `c.sra`) junto con la compuerta `c.xor`.

None

```
00200513 // addi a0, x0, 2 - a0 = 2
00300593 // addi a1, x0, 3 - a1 = 3
00b54533 // xor a0, a0, a1 - xor clásica: a0 = 2 ^ 3 = 1
05328da9 // [c.slli a0, 12] | [c.xor a1, a0] - c.xor: a1 = 3 ^ 1 = 2 //
c.slli: a0 = 1 << 12 = 4096 (0x1000)
00255513 // srli a0, a0, 2 - srl clásica: a0 = 4096 >> 2 = 1024 (0x400)
812d0585 // [c.srli a0, 11] | [c.addi a1, 1] - c.addi: a1 = 2 + 1 = 3 //
c.srli: a0 = 1024 >> 11 = 0
```

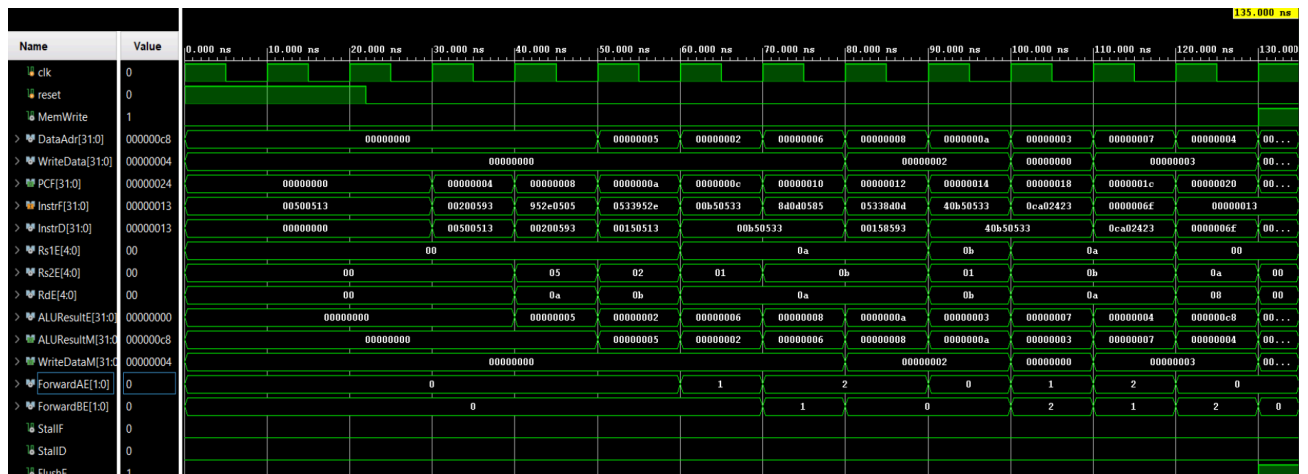
```
00b54533 // xor a0, a0, a1 - xor clásica:  $a0 = 0 \wedge 3 = 3$ 
40155513 // srai a0, a0, 1 - sra clásica:  $a0 = 3 \ggg 1 = 1$ 
85050505 // [c.srai a0, 1] | [c.addi a0, 1] - c.addi:  $a0 = 1 + 1 = 2$  //
c.srai:  $a0 = 2 \ggg 1 = 1$ 
0ca02423 // sw a0, 200(x0) - valor final: 1
0000006f // jal x0, 0
```

4. **Prueba LUI y General:** Un bloque ininterrumpido que encadena todas las operaciones anteriores e introduce la instrucción `c.lui` (Load Upper Immediate comprimido) para verificar la carga de constantes en los bits superiores..

```
None
00100593 // addi a1, x0, 1
8d4d6505 // [c.or a0, a1] | [c.lui a0, 1]
00154513 // xori a0, a0, 1
952e0585 // [c.add a0, a1] | [c.addi a1, 1]
40b50533 // sub a0, a0, a1
8d6d8da9 // [c.and a0, a1] | [c.xor a1, a0]
00255513 // srli a0, a0, 2
050a8129 // [c.slli a0, 2] | [c.srli a0, 10]
0ca02423 // sw a0, 200(x0)
0000006f // jal x0, 0
```

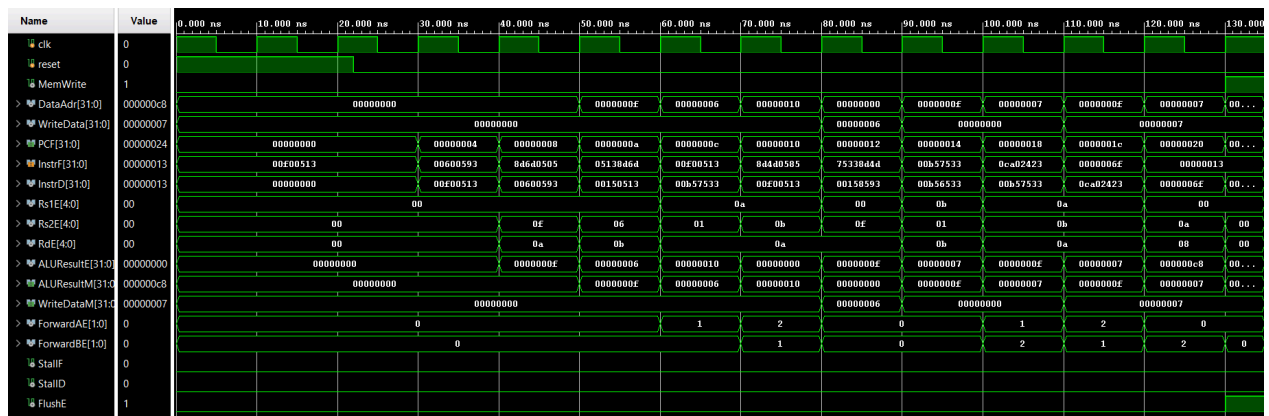
2.2 Validación de cada una de las instrucciones

Validacion de Prueba Aritmética (c.addi, c.add, c.sub):



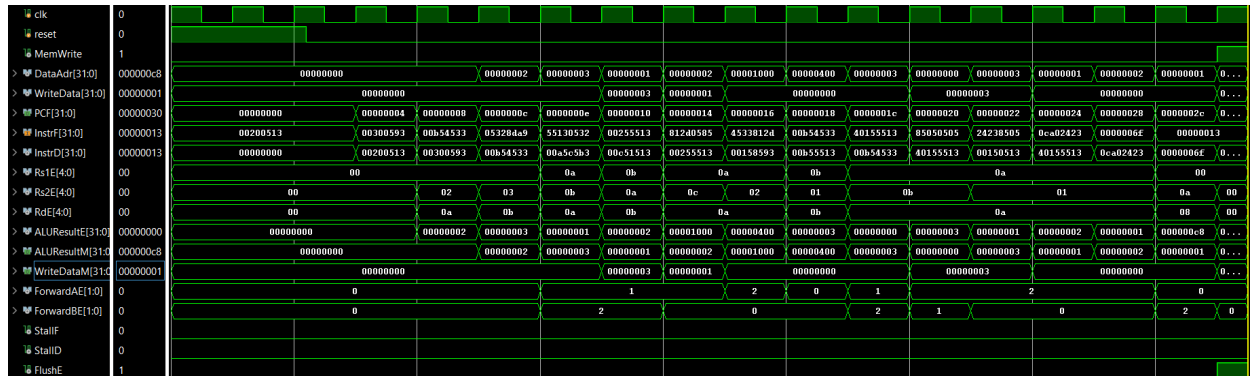
La simulación confirma el dinamismo del hardware híbrido al observar la señal PCF, la cual arranca con saltos de 4 bytes (0, 4, 8) para las instrucciones base de 32 bits y cambia instantáneamente a saltos de 2 bytes (8, a, c, e) al decodificar las instrucciones comprimidas (RVC). A nivel aritmético, la señal ALUResultE demuestra una ejecución matemática exacta y continua: el procesador carga los valores iniciales 5 y 2, los escala a 6 y 8 mediante sumas empaquetadas de 16 bits, alcanza el pico de el valor a (10 en decimal) con una suma de 32 bits, y desciende correctamente a través de restas pasando por 3 y 7 hasta llegar a 4. El bloque finaliza su ejecución levantando la señal MemWrite a 1 para escribir definitivamente el dato 00000004 como se puede observar en la señal WriteData en y se guarda en la dirección de memoria 000000c8 como se puede ver en la señal DataAdr.

Validacion de Prueba Logica (c.and, c.or):



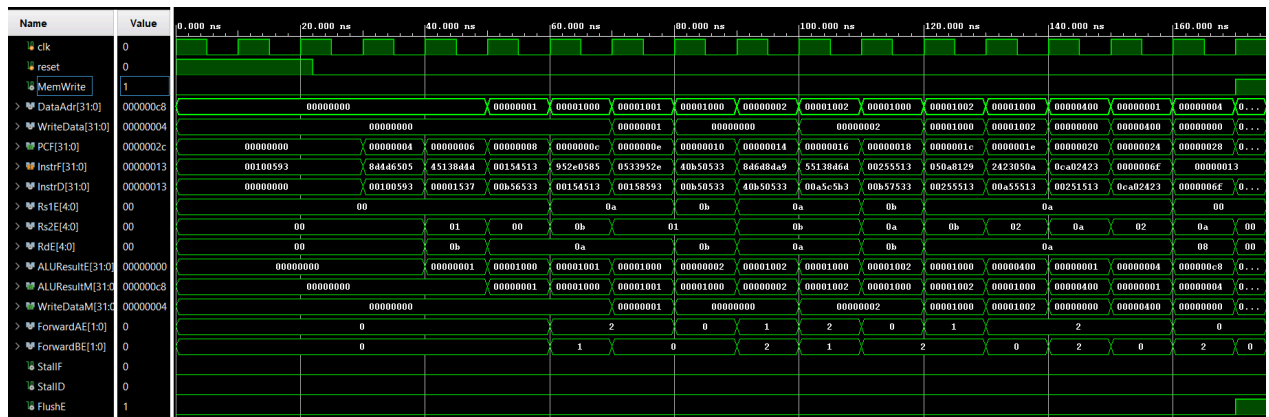
Esta ejecución valida el procesamiento de operaciones lógicas bit a bit en la ALU, intercalando anchos de instrucción de manera fluida. Al analizar la propagación, se observa cómo el código híbrido transiciona sin errores desde la etapa de *Fetch* (InstrF) hacia *Decode* (InstrD). La traza de ALUResultE refleja la evolución de los datos iniciados en f (15) y 6: primero, la compuerta c.and comprimida procesa los bits llevando el resultado temporal a 0, para luego aplicar funciones OR y AND consecutivas operando con los valores f y 7. El aspecto más destacable de este *waveform* es la intervención de la Unidad de Riesgos mediante las señales ForwardAE y ForwardBE. Debido a la dependencia inmediata de los datos en este bloque tan denso, estas señales se activan (mostrando valores de 1 o 2) para "adelantar" los resultados recién calculados directamente hacia los multiplexores de la ALU. Esto garantiza que el flujo de ejecución no sufra congelamientos (*stalls*). Además, se verifica cómo el identificador del registro destino viaja protegido a través de la señal RdE, culminando la rutina al transferir el valor final hacia la etapa de memoria (ALUResultM) y almacenando de forma limpia el dato 00000007 como se puede verificar en la señal WriteData.

Validación Prueba de Desplazamientos y XOR:



El análisis de este waveform evalúa el rendimiento de los desplazadores bidireccionales y las operaciones exclusivas (XOR) bajo una alta densidad de código. Al rastrear el flujo de las instrucciones desde la etapa de *Fetch* (InstrF) hacia *Decode* (InstrD), la señal ALUResultE captura una intensa manipulación a nivel de bits: tras inicializar los registros en 2 y 3, una compuerta XOR clásica genera un 1, el cual es inmediatamente sometido a un corrimiento lógico a la izquierda de 12 posiciones (c.slli) que dispara el valor a 1000 (4096 en hexadecimal). Posteriormente, los desplazamientos lógicos hacia la derecha (srli y c.srli) reducen drásticamente la cifra, pasando por 400 hasta vaciarla de forma temporal. Debido a la fuerte dependencia de datos encadenados en este bloque, la Unidad de Riesgos interviene de manera agresiva; las señales ForwardAE y ForwardBE registran múltiples activaciones (conmutando entre estados 1 y 2) para inyectar los valores recién calculados directamente en los multiplexores de la ALU. Este puenteo de hardware (*forwarding*) es tan efectivo que evade por completo la necesidad de congelar el *pipeline* (las señales StallF y StallD se mantienen intactas en 0). Finalmente, el datapath rastrea con éxito los registros destino a través de la señal RdE (alternando entre 0a y 0b), conduciendo el dato final hacia la etapa de memoria (ALUResultM) para concluir la ejecución almacenando de forma impecable el valor 00000001 como se puede ver en la señal WriteData y en la dirección 000000c8 se guarda correctamente como se puede ver en la señal de DataAdr.

Validación Prueba LUI y General:



Este último waveform consolida la validación total de la arquitectura RV32IC sometiendo al datapath a un estrés de dependencias continuas sin utilizar instrucciones de relleno (NOPs). Al observar el tránsito desde la fase de búsqueda (InstrF) hacia la decodificación (InstrD), destaca la ejecución exitosa de la carga superior comprimida (c.lui), la cual se refleja de inmediato en ALUResultE inyectando el valor 00001000 en la parte alta del registro. A partir de ahí, la gráfica muestra una mutación aritmética implacable ciclo a ciclo: el dato escala a 1001 y 1002, se desplaza hacia la derecha reduciéndose a 00000400 (1024), cae a 1 y vuelve a escalar a la izquierda hasta su forma definitiva de 00000004. La extrema densidad de este código genera colisiones de datos inminentes; sin embargo, la Unidad de Riesgos demuestra una eficiencia impecable. Las señales de forwarding (ForwardAE y ForwardBE) operan a máxima capacidad alternando constantemente entre los estados 1 y 2 para puentear los operandos directamente hacia la ALU. Gracias a esta resolución dinámica, el procesador evade completamente la necesidad de detenerse (las señales StallF y StallD se mantienen planchadas en 0 durante toda la ejecución). Finalmente, con el rastro de los registros destino alternando correctamente en RdE (0a y 0b), el ciclo culmina transfiriendo el dato hacia WriteDataM para escribir de manera definitiva el valor 00000004 como se puede ver en la señal WriteData y en la dirección 000000c8 se guarda correctamente como se puede ver en la señal de DataAdr.

3. Bibliografía

- I. Harris, D., & Harris, S. (2020). *Digital Design and Computer Architecture: RISC-V Edition*. Morgan Kaufmann.
- II. RISC-V Collaboration. (s.f.). riscv-gnu-toolchain: GNU toolchain for RISC-V, including GCC. GitHub. Recuperado el 23 de junio de 2026, de <https://github.com/riscv-collab/riscv-gnu-toolchain>